

Task 2 – Test Plan

The plan tests the fitness tracker through its graphical interface using black-box testing: I enter inputs, then compare what the program displays with the result I expected. I selected five functions: (1) registration validation, (2) login with lockout, (3) recording activities and calculating calories, (4) goal setting and goal reporting, and (5) data loading and saving. The first three are the minimum required; the others cover the remaining functional requirements. Each function has valid, invalid and boundary inputs, and exception tests check that errors are handled without crashing.

Test environment

Tests run on Windows using the Debug build of the project from Visual Studio. The program stores data in users.json beside the executable, so this file, its backup and the error log are the only external resources involved. Because the application is a Windows Forms program, tests are run manually and documented with screenshots. Each group states its precondition so that results can be repeated exactly.

Purpose of each test group

Registration (R1–R10) checks that usernames contain only letters and numbers and that passwords are exactly twelve characters with at least one lowercase and one uppercase letter. It also checks that duplicate usernames are refused and that helpful messages are shown.

Login (L1–L8) checks that only correct credentials give access, that failure messages do not reveal whether the username exists, and that three failed attempts trigger a thirty-second lockout, including when the correct password is entered during the lockout.

Activity recording (C1–C13) checks that each of the six activities accepts three metrics and calculates calories correctly. Expected values are calculated by hand from the formulas in the source code. Invalid entries (zero, negative, text, blank) and extreme values are included to confirm they are rejected.

Goal tests (G1–G7) check that a positive goal is accepted, invalid goals are rejected, the goal is remembered after logging out and restarting, the total is summed correctly, and the achieved or remaining status is correct, including the exact boundary where total equals goal.

Data tests (D1–D3) check exception handling: a missing users.json, a corrupted users.json with a valid backup, and a corrupted file with no backup.

Test scripts and results

Each test follows the same script: start from the stated precondition, perform the steps, and compare the outcome with the expected result. A test passes only if the displayed message or value matches the expectation and the program keeps running. The results table records the actual outcome and a pass or fail status. Any failure is described, fixed, and retested, and the retest result is recorded. Screenshots of each test are kept as evidence in the appendix.

Task 2 – Justification of Test Data Selection and Execution

How the data was selected. I chose the test data by asking what a user could type into each screen: something correct, something wrong, and something right on the limit. Every function has at least one of each, so I can see that the program accepts good input and refuses bad input with a helpful message.

Registration. The brief sets clear rules, so each rule has its own test. A username with a space or a symbol breaks the letters-and-numbers rule. Passwords of 11 and 13 characters sit either side of the required 12, and passwords with no capital letter or no lowercase letter break the other two rules. I also tried the same username twice, because two users must not share one.

Login. I used a correct password, a wrong password, an unknown username and empty boxes. I entered three wrong passwords in a row because the program locks the user out at that point, and I then tried the correct password during the lockout to check that it is still refused.

Activities. I used simple whole numbers, such as 5 km or 30 minutes, so I could work out the calories by hand before running each test. Working the answer out first means the expected result does not come from the program itself. I then tried zero, a negative number, letters and an empty box, because these are the mistakes a user is most likely to make. I also tried one extremely large number to see whether the program copes with it.

Goals. I used a valid goal, invalid goals and a goal exactly equal to the total calories, because that is the point where the message should change to "achieved".

Data file. I deleted the users.json file and damaged it on purpose, because a missing or broken file is the most likely problem the program will meet when it starts.

How the tests were run. I ran the program from Visual Studio and tested one function at a time, in the order shown in the tables. The order matters because some tests need earlier ones: an account must exist before it can log in, and activities must be recorded before the total can be checked. I started with an empty users.json so the results are the same every time the tests are repeated. For each test I typed the input, wrote down what the program showed, compared it with the expected result, and took a screenshot as evidence. For the goal and data tests I closed and reopened the program to check that the information had really been saved. Any test that failed was fixed and repeated.

Limits of the testing. The tests were run by hand, so they need to be repeated after any change to the code. The calorie formulas are estimates, so the tests only check that the program follows its own formulas and not that the results are medically exact.

Task 3 – Justification of Class Design

The program uses five classes, each with one clear responsibility, so that data, storage and interface concerns stay separate and can change independently.

User represents an account and owns everything belonging to it: the username, the calorie goal and the list of recorded activities. Keeping the goal inside User is what connects it to the username so it is remembered between sessions, as the brief requires. Encapsulation is applied to the password: only a SHA-256 hash (PasswordHash) is stored, and the hashing routine ComputeHash is private. Other classes can only call SetPassword and VerifyPassword, so no plain-text password is stored or exposed, and the hashing method could be replaced without touching any other class.

ActivityRecord represents one recorded session: the activity name, the three metric values, the calculated calories and a timestamp. One class serves all six activities because every activity has exactly three metrics; only their meaning and the formula differ, and these are selected by activity name. Six near-identical classes would add duplication without adding behaviour. Storing the calculated calories with each record means the total is a simple sum and old records do not change if a formula is later adjusted. The relationship is composition: a User contains zero or many ActivityRecords, which have no meaning outside their user.

DataStore is responsible only for persistence. It holds the list of users and provides Load and Save using JSON. Isolating file access here means the interface never handles files directly. It also concentrates error handling: loaded data is validated, a backup is made before each save, the backup is used if the file is corrupted, and errors are logged, so a damaged file does not crash the program. Helper methods such as TryRecoverFromBackup and LogError are private.

Form1 is the presentation and control class. It manages the login and dashboard panels, validates usernames and passwords, applies a 30-second lockout after three failed logins, and calls CalculateCalories, which uses a selection structure to apply the formula for each of the six activities. Its state (current user, failed-attempt counter, lockout time) is private. It uses User and DataStore instead of duplicating their logic, so dependencies run one way: interface, then model, then storage. Program is only the entry point that starts Form1.

Design trade-offs: this layered structure keeps the class count suitable for a small application and makes each part easy to read and test. I recognise that calorie calculation and validation currently sit inside Form1. In a larger system I would move them into a CalorieCalculator class, or create an abstract Activity class with six subclasses overriding a Calculate method, giving polymorphism and easier extension. I chose the simpler approach because the six activities share an identical structure, so inheritance would add classes without adding new behaviour.

Appendix A – Test Scripts and Results

Registration (R1–R10)

Precondition: no users.json exists.

ID	Input (username / password)	Expected result	Actual result	Pass/Fail
R1	john123 / MyPassword01 → Register	Green "Registration successful" message		
R2	john 123 / MyPassword01	Red error: letters and numbers only; no account created		
R3	john_123 / MyPassword01	Same error as R2		
R4	Both fields left untouched (placeholder text)	Red error shown; no account created		
R5	john456 / MyPassword0 (11 characters)	Error: password must be exactly 12 characters		
R6	john456 / MyPassword012 (13 characters)	Same error as R5		
R7	john456 / mypassword01 (no uppercase)	Same error as R5		
R8	john456 / MYPASSWORD01 (no lowercase)	Same error as R5		
R9	JOHN123 / MyPassword01	"Username already taken"		
R10	12345 / Abcdefghijkl	Registration successful (digits-only username is valid)		

Login (L1–L8)

Precondition: john123 is registered. Restart the application before L2, L6 and L7 so the failed-attempt counter starts at zero.

ID	Input (username / password)	Expected result	Actual result	Pass/Fail
L1	john123 / MyPassword01	Dashboard opens: "Welcome, john123"		
L2	john123 / WrongPassword1	"Invalid credentials", 2 attempts remaining		
L3	nouser99 / MyPassword01 (same session as L2)	Same message, 1 attempt remaining (does not reveal that the user is unknown)		
L4	Blank username	"Please enter your username"; no attempt counted		
L5	JOHN123 / MyPassword01	Login succeeds (username is not case-sensitive)		
L6	john123 / mypassword01	Login refused (password is case-sensitive)		

ID	Input (username / password)	Expected result	Actual result	Pass/Fail
L7	Wrong password three times in a row	Counts down 2, 1, then "Account locked for 30 seconds"		
L8	During lockout enter correct credentials; retry after 30 seconds	Refused with "Try again in N seconds"; after 30 seconds login succeeds		

Activity recording (C1–C13)

Precondition: logged in as john123. Metrics are entered in the order shown on screen.

ID	Activity and metrics	Expected result	Actual result	Pass/Fail
C1	Walking: 5000 / 4 / 50	440.0 kcal		
C2	Swimming: 20 / 25 / 30	252.0 kcal		
C3	Running: 5 / 30 / 150	307.5 kcal		
C4	Cycling: 20 / 60 / 20	610.0 kcal		
C5	Rowing: 1000 / 3 / 20	152.0 kcal		
C6	Yoga: 60 / 5 / 180	180.0 kcal		
C7	Yoga: 60 / 10 / 180	360.0 kcal (intensity doubles the result)		
C8	Yoga: 60 / 2 / 180	180.0 kcal (multiplier stays at 1)		
C9	Walking: 0 / 4 / 50	"Invalid Metric 1" error; no record added		
C10	Walking: 5000 / -4 / 50	"Invalid Metric 2" error		
C11	Walking: 5000 / 4 / abc	"Invalid Metric 3" error		
C12	Walking: (blank) / 4 / 50	"Invalid Metric 1" error		
C13	Walking: 1E308 / 1E308 / 1	Rejected with an error message; no crash and no record added		

Goal setting and reporting (G1–G7)

Precondition: register goaltest / MyPassword01 and log in. G6 and G7 use the six activities C1–C6 only.

ID	Input	Expected result	Actual result	Pass/Fail
G1	Goal 300 → Save Goal	"Daily goal set to 300.0 kcal"		
G2	Goal 0	"Please enter a valid positive number"		
G3	Goal -50	Same error as G2		
G4	Goal abc	Same error as G2		
G5	Log out, close and reopen, log in	Goal box still shows 300		

ID	Input	Expected result	Actual result	Pass/Fail
G6	Enter C1–C6 activities, set goal 2000	Total 1941.5 kcal; "58.5 kcal remaining"		
G7	Set goal 1941.5	Status "ACHIEVED" (total equals goal)		

Data loading and saving (D1–D3)

Precondition: at least one account has been saved so that a backup file exists.

ID	Action	Expected result	Actual result	Pass/Fail
D1	Close the app, delete users.json, reopen	Login screen appears; registering works and a new users.json is created		
D2	Replace users.json contents with { "Users": [(backup exists), reopen	App opens; earlier accounts recovered from backup; entry added to error log		
D3	Corrupt users.json and delete the backup, reopen	App opens with no accounts and does not crash		